EX NO: 1 Date:	Write a Java Program to Print Fibonacci Series Using Loops

Aim:

To Write a Java Program to Print Fibonacci Series Using Loops

Algorithm :

Step 1: Declare integer variables n, first, second, and next.

Initialize first = 0 and second = 1.

Step 2: Create a Scanner object to read input from the user.

Step 3: Prompt the user to enter the number of terms required in the Fibonacci series.

Step 4: Read the value of n.

Step 5: Check whether n is less than or equal to 0.

- If yes, print "Invalid input" and terminate the program.

Step 6: If n == 1, print first and terminate.

Step 7: Otherwise, print first and second as the first two terms.

Step 8: Use a for loop from i = 3 to n.

Step 9: Inside the loop:

- Calculate next = first + second.
- Print next.
- Assign first = second.
- Assign second = next.

Step 10: End the loop when all terms are printed.

Step 11: Stop the program.

Program:

```
import java.io.*;
import java.util.Scanner;

class Fibonacci {

   public static void main(String[] args) {

      int n, first = 0, second = 1, next;

      Scanner sc = new Scanner(System.in);


      System.out.print("Enter number of terms: ");

      n = sc.nextInt();


      System.out.println("Fibonacci Series:");

      System.out.print(first + " " + second + " ");


      for(int i = 3; i <= n; i++) {

         next = first + second;

         System.out.print(next + " ");

         first = second;

         second = next;

      }

   }

}
```

Output:

Enter number of terms: 7

Fibonacci Series: 0 1 1 2 3 5 8

RESULT:

Thus, the Java program to print Fibonacci series using loops was executed successfully.

EX NO: 2 DATE:	**Write a Java program to check whether a number is prime or not.**

Aim:

To write a Java program to check whether a number is prime or not.

Algorithm:

Step 1: Start the program.

Step 2: Declare variables n, i, flag = 0.

Step 3: Create Scanner object.

Step 4: Read the number n.

Step 5: If n <= 1, it is not prime.

Step 6: Use for loop from 2 to n/2.

Step 7: If n % i == 0, set flag = 1.

Step 8: Break the loop.

Step 9: If flag == 0, print prime.

Step 10: Stop the program.

Program:

```
import java.util.Scanner;

class Prime {
   public static void main(String args[]) {
      int n, flag = 0;
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter a number: ");
      n = sc.nextInt();

      for(int i = 2; i <= n/2; i++) {
         if(n % i == 0) {
            flag = 1;
            break;
         }
      }

      if(n <= 1)
         System.out.println("Not Prime");
      else if(flag == 0)
         System.out.println("Prime Number");
      else
         System.out.println("Not Prime");
   }
}
```

Output:

Enter a number: 7

Prime Number

Result:
Thus, the Java program to check prime number was executed successfully.

EX NO: 3 Date:	**Write a Java program to find factorial of a number using recursion.**

Aim:

To write a Java program to find factorial of a number using recursion.

Algorithm:

Step 1: Start the program.

Step 2: Define recursive function factorial(n).

Step 3: If n == 0 or 1, return 1.

Step 4: Else return n * factorial(n-1).

Step 5: Create Scanner object.

Step 6: Read number n.

Step 7: Call factorial(n).

Step 8: Store result.

Step 9: Display result.

Step 10: Stop the program.

Program:

```
import java.util.Scanner;


class Factorial {
   static int fact(int n) {
      if(n == 0 || n == 1)
         return 1;
      else
         return n * fact(n - 1);
   }

   public static void main(String args[]) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number: ");
      int n = sc.nextInt();

      int result = fact(n);
      System.out.println("Factorial = " + result);
   }
}
```

Output:

Enter a number: 5

Factorial = 120

Result:

Thus, the Java program to find factorial using recursion was executed successfully.

EX NO:4 DATE:	**write a Java program to demonstrate conditional statements and loops.**

Aim:

To write a Java program to demonstrate conditional statements and loops.

Algorithm:

Step 1: Start the program.

Step 2: Declare variable n.

Step 3: Create Scanner object.

Step 4: Read number n.

Step 5: Check if n is positive or negative using if-else.

Step 6: Use for loop from 1 to n.

Step 7: Print numbers.

Step 8: End loop.

Step 9: Display message.

Step 10: Stop program.

Program:

```
import java.util.Scanner;

class ConditionLoop {
   public static void main(String args[]) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number: ");
      int n = sc.nextInt();

      if(n > 0)
         System.out.println("Positive Number");
      else
         System.out.println("Negative Number");

      System.out.println("Numbers from 1 to " + n);

      for(int i = 1; i <= n; i++) {
         System.out.print(i + " ");
      }
   }
}
```

Output:

Enter a number: 5

Positive Number
Numbers from 1 to 5
1 2 3 4 5

Result:

Thus, the Java program demonstrating conditional statements and loops was executed successfully.

EX NO: 5 DATE:	write a Java program to demonstrate 1D and 2D array operations.

Aim:

To write a Java program to demonstrate 1D and 2D array operations.

Algorithm:

Step 1: Start the program.

Step 2: Declare 1D array.

Step 3: Read elements.

Step 4: Display elements.

Step 5: Declare 2D array.

Step 6: Read 2D elements.

Step 7: Use nested loop.

Step 8: Display 2D array.

Step 9: End loops.

Step 10: Stop program.

Program:

```
import java.io.*;
import java.util.Scanner;


class ArrayDemo {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);


        int arr[] = new int[3];
        System.out.println("Enter 3 elements:");
        for(int i = 0; i < 3; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.println("1D Array:");
        for(int i = 0; i < 3; i++) {
            System.out.print(arr[i] + " ");
        }

        int mat[][] = new int[2][2];
        System.out.println("\nEnter 4 elements for 2D array:");
        for(int i = 0; i < 2; i++) {
            for(int j = 0; j < 2; j++) {
                mat[i][j] = sc.nextInt();
            }
        }

        System.out.println("2D Array:");
        for(int i = 0; i < 2; i++) {
            for(int j = 0; j < 2; j++) {
```

```
                System.out.print(mat[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

Output:

```
1 2 3
4 5 6 7

1D Array:
1 2 3
2D Array:
4 5
6 7
```

Result:

Thus, the Java program to demonstrate 1D and 2D arrays was executed successfully.

EX NO:6 DATE:	write a Java program to demonstrate classes and objects.

Aim:

To write a Java program to demonstrate classes and objects.

Algorithm:

Step 1: Start the program.

Step 2: Define a class Student.

Step 3: Declare data members (name, roll).

Step 4: Create a method display().

Step 5: Create main class.

Step 6: Create object of Student class.

Step 7: Assign values.

Step 8: Call display() method.

Step 9: Print details.

Step 10: Stop the program.

Program:

```
import java.io.*;
import java.util.Random;

class Student {
   String name;
   int roll;


   void display() {
      System.out.println("Name: " + name);
      System.out.println("Roll No: " + roll);
   }
}


class MainClass {
   public static void main(String args[]) {
      Student s1 = new Student();
      s1.name = "KUMAR";
      s1.roll = 101;
      s1.display();
   }
}
```

Output:

Name: KUMAR

Roll No: 101

Result:

Thus, the Java program to demonstrate classes and objects was executed successfully.

EX NO:7 DATE:	write a Java program to implement constructor overloading.

Aim:

To write a Java program to implement constructor overloading.

Algorithm:

Step 1: Start program.

Step 2: Create class Sample.

Step 3: Define default constructor.

Step 4: Define parameterized constructor.

Step 5: Print values inside constructors.

Step 6: Create main method.

Step 7: Create object using default constructor.

Step 8: Create object using parameterized constructor.

Step 9: Display results.

Step 10: Stop program.

Program:

```
Import java.io.*;
class Sample {
   Sample() {
      System.out.println("Default Constructor");
   }

   Sample(int a) {
      System.out.println("Parameterized Constructor: " + a);
   }

   public static void main(String args[]) {
      Sample s1 = new Sample();
      Sample s2 = new Sample(10);
   }
}
```

Output:

Default Constructor
Parameterized Constructor: 10

Result:

Thus, constructor overloading was implemented successfully.

EX NO:8 DATE:	**write a Java program to implement method overloading and overriding.**

AIM:

To write a Java program to implement method overloading and overriding.

Algorithm:

Step 1: Start program.

Step 2: Create class A with method show().

Step 3: Overload method show(int x).

Step 4: Create class B extends A.

Step 5: Override method show().

Step 6: Create main method.

Step 7: Create object of B.

Step 8: Call show() method.

Step 9: Call overloaded method.

Step 10: Stop program.

Program:

```
Import java.io.*;

class A {
   void show() {
      System.out.println("Parent Class Method");
   }

   void show(int x) {
      System.out.println("Overloaded Method: " + x);
   }
}

class B extends A {
   void show() {
      System.out.println("Child Class Overridden Method");
   }

   public static void main(String args[]) {
      B obj = new B();
      obj.show();
      obj.show(5);
   }
}
```

Output:

Child Class Overridden Method
Overloaded Method: 5

Result:

Thus, method overloading and overriding were implemented successfully.

EX NO: 9 DATE:	**Demonstrate use of this, super, final and static keywords.**

Aim:

To demonstrate use of this, super, final and static keywords.

Algorithm:

Step 1: Start program.

Step 2: Create parent class with constructor.

Step 3: Create child class extends parent.

Step 4: Use super keyword.

Step 5: Declare static variable.

Step 6: Declare final variable.

Step 7: Use this keyword.

Step 8: Create object.

Step 9: Display values.

Step 10: Stop program.

Program:

```
import java.util.Scanner;

class Parent {
   Parent() {
      System.out.println("Parent Constructor");
   }
}

class Child extends Parent {
   static int x = 100;
   final int y = 200;

   Child(int x) {
      super();
      this.display(x);
   }

   void display(int x) {
      System.out.println("Static: " + Child.x);
      System.out.println("Final: " + y);
      System.out.println("This value: " + x);
   }

   public static void main(String args[]) {
      Child obj = new Child(50);
   }
}
```

Sample Output:

Parent Constructor
Static: 100
Final: 200
This value: 50

Result:

Thus, this, super, final, and static keywords were demonstrated successfully.

EX NO: 10 DATE:	**Implement single and multilevel inheritance.**

Aim:

To implement single and multilevel inheritance.

Algorithm:

Step 1: Start program.

Step 2: Create class A.

Step 3: Create class B extends A.

Step 4: Create class C extends B.

Step 5: Define methods in each class.

Step 6: Create main method.

Step 7: Create object of C.

Step 8: Call methods.

Step 9: Display output.

Step 10: Stop program.

Program:

```
class A {
   void displayA() {
      System.out.println("Class A");
   }
}

class B extends A {
   void displayB() {
      System.out.println("Class B");
   }
}

class C extends B {
   void displayC() {
      System.out.println("Class C");
   }

   public static void main(String args[]) {
      C obj = new C();
      obj.displayA();
      obj.displayB();
      obj.displayC();
   }
}
```

Sample Output:

Class A
Class B
Class C

Result:
Thus, single and multilevel inheritance were implemented successfully.

EX NO:11 DATE:	Implement abstract class and interface.

Aim:

To implement abstract class and interface.

Algorithm:

Step 1: Start program.

Step 2: Create abstract class.

Step 3: Declare abstract method.

Step 4: Create interface.

Step 5: Declare interface method.

Step 6: Create class implementing both.

Step 7: Define methods.

Step 8: Create object.

Step 9: Call methods.

Step 10: Stop program.

Program:

```
import java.io.*;
import java.util.Random;

abstract class Shape {
   abstract void draw();
}

interface Color {
   void fill();
}

class Circle extends Shape implements Color {
   void draw() {
      System.out.println("Drawing Circle");
   }

   public void fill() {
      System.out.println("Filling Color");
   }

   public static void main(String args[]) {
      Circle obj = new Circle();
      obj.draw();
      obj.fill();
   }
}
```

Output:

Drawing Circle
Filling Color

Result:

Thus, abstract class and interface were implemented successfully.

EX NO:12 DATE:	**Implement multiple inheritance using interfaces.**

Aim:

To implement multiple inheritance using interfaces.

Algorithm:

Step 1: Start program.

Step 2: Create interface A.

Step 3: Create interface B.

Step 4: Declare methods.

Step 5: Create class implementing both interfaces.

Step 6: Define methods.

Step 7: Create object.

Step 8: Call methods.

Step 9: Display output.

Step 10: Stop program.

Program:

```
import java.io.*;

interface A {
   void showA();
}

interface B {
   void showB();
}

class Test implements A, B {
   public void showA() {
      System.out.println("Interface A");
   }

   public void showB() {
      System.out.println("Interface B");
   }

   public static void main(String args[]) {
      Test obj = new Test();
      obj.showA();
      obj.showB();
   }
}
```

Output:

Interface A
Interface B

Result:

Thus, multiple inheritance using interfaces was implemented successfully.

EX NO:13 DATE:	Demonstrate exception handling using try-catch-finally.

AIM:

To demonstrate exception handling using try-catch-finally.

Algorithm:

Step 1: Start program.

Step 2: Declare variables.

Step 3: Use try block.

Step 4: Write risky code.

Step 5: Use catch block.

Step 6: Handle exception.

Step 7: Use finally block.

Step 8: Print message.

Step 9: End blocks.

Step 10: Stop program.

Program:

```
ExceptionDemo {
   public static void main(String args[]) {
      try {
         int a = 10 / 0;
      }
      catch(ArithmeticException e) {
         System.out.println("Exception Caught");
      }
      finally {
         System.out.println("Finally Block Executed");
      }
   }
}
```

Output:

Exception Caught
Finally Block Executed

Result:

Thus, exception handling was demonstrated successfully.

EX NO: 14 DATE:	Create and throw a user-defined exception.

Aim:

To create and throw a user-defined exception..

Algorithm:

Step 1: Start program.

Step 2: Create custom exception class.

Step 3: Extend Exception class.

Step 4: Define constructor.

Step 5: Create main class.

Step 6: Check condition.

Step 7: Throw exception.

Step 8: Catch exception.

Step 9: Display message.

Step 10: Stop program.

Program:

```
class MyException extends Exception {
   MyException(String msg) {
      super(msg);
   }
}

class TestException {
   public static void main(String args[]) {
      try {
         int age = 15;
         if(age < 18)
            throw new MyException("Not Eligible to Vote");
      }
      catch(MyException e) {
         System.out.println(e.getMessage());
      }
   }
}
```

Sample Output:

Not Eligible to Vote

Result:

Thus, user-defined exception was created and handled successfully.

EX NO: 15 DATE:	Create a file, write data, and read data using Java.

Aim:

To create a file, write data, and read data using Java.

Algorithm:

Step 1: Start program.

Step 2: Import io package.

Step 3: Create FileWriter object.

Step 4: Write data.

Step 5: Close writer.

Step 6: Create FileReader.

Step 7: Read data.

Step 8: Display data.

Step 9: Close reader.

Step 10: Stop program.

Program:

```
import java.io.*;

class FileDemo {
   public static void main(String args[]) throws Exception {
      FileWriter fw = new FileWriter("sample.txt");
      fw.write("Hello Java File Handling");
      fw.close();

      FileReader fr = new FileReader("sample.txt");
      int i;
      while((i = fr.read()) != -1) {
         System.out.print((char)i);
      }
      fr.close();
   }
}
```

Sample Output:

Hello Java File Handling

Result:
Thus, the Java program for file creation, writing and reading was executed successfully